

Figure 1: Distributed computing environment

needs, and then compile them using the gcc compiler as shown in Figure 5.

```
/* This is dist_app_client.c generated by rpcgen and
modified according to the need */
```

```
#include "dist_app.h"
void minmax_1(char *host1, char *host2, int *p, int length) //
Client Program has to communicate
{
    // with two Computers as per Figure 2.
    CLIENT *clnt1, *clnt2;
    int *result_1, i;
    struct ipair op;
    int *result_2;
    for(i=0; i<length; i++) //making data ready
        op.data[i] = p[i];
        op.count = length;

#ifdef DEBUG
    clnt1 = clnt_create(host1, MINMAX, VER1, "udp"); //
for making UDP connection
    if (clnt1 == NULL) {
        clnt_pcreateerror(host1);
        exit(1);
    }
    clnt2 = clnt_create(host2, MINMAX, VER1, "udp"); // for
making UDP Connection
    if (clnt2 == NULL) {
        clnt_pcreateerror(host2);
        exit(1);
    }
#endif /* DEBUG */

    result_1 = max_1(&op, clnt1); // sending data to
service Max as per Figure 2
    if (result_1 == (int *) NULL) {
        clnt_perror(clnt1, "call failed");
    }
    printf("\n The result from the Service Max is
```

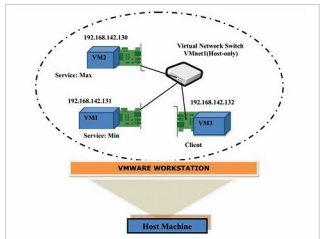


Figure 2: Test bed

```
root@client-virtual-machine:/home/client/RPC#
root@client-virtual-machine:/home/client/RPC# rpcgen -C dist_app.x
root@client-virtual-machine:/home/client/RPC# ls
dist_app_client.c dist_app.h dist_app_svc.c dist_app.x dist_app_xdr.c
```

Figure 3: Compiling the IDL file with *rpcgen*

```
root@client-virtual-machine:/home/client/Desktop/RPC#
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -c dist_app.x -o dist_app.o
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -c dist_app_client.c -o dist_app_client.o
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -c dist_app_svc.c -o dist_app_svc.o
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -c dist_app_xdr.c -o dist_app_xdr.o
root@client-virtual-machine:/home/client/Desktop/RPC# gcc dist_app.o dist_app_client.o dist_app_svc.o dist_app_xdr.o -o dist_app
```

Figure 4: Creating sample client and sample server programs with *rpcgen*

```
root@client-virtual-machine:/home/client/Desktop/RPC#
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -o client dist_app.o dist_app.o -o dist_app.o
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -o service_max dist_app.o dist_app.o -o dist_app.o
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -o service_min dist_app.o dist_app.o -o dist_app.o
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -o client dist_app.o dist_app.o -o dist_app.o
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -o service_max dist_app.o dist_app.o -o dist_app.o
root@client-virtual-machine:/home/client/Desktop/RPC# gcc -o service_min dist_app.o dist_app.o -o dist_app.o
```

Figure 5: Generating a distributed application with gcc

```
%d\n", *result_1);
    result_2 = min_1(&op, clnt2); // sending data to
service Min as per Figure 2
    if (result_2 == (int *) NULL) {
        clnt_perror(clnt2, "call failed");
    }
    printf("\n The result from the Service Min is
%d\n", *result_2);
#ifdef DEBUG
    clnt_destroy(clnt1);
    clnt_destroy(clnt2);
#endif /* DEBUG */
}
int main (int argc, char *argv[])
{
    char *host1, *host2;

    int i, x[10];
    printf("\n give any 10 elements"); // For
collecting the list of elements from USER
    for(i=0; i<10; i++)
        scanf("%d", &x[i]);
```